

SQL (Structured Query Language) :

(mysql- rdbms)

SQL, or Structured Query Language, is a domain-specific language used in programming and **managing relational databases**. It provides a standardized way of interacting with databases, allowing users to perform various operations such as querying, updating, inserting, and deleting data.

Table

What is a Table?

- A table is like a spreadsheet. **It has rows and columns where we can store information.**

Rows and Columns:

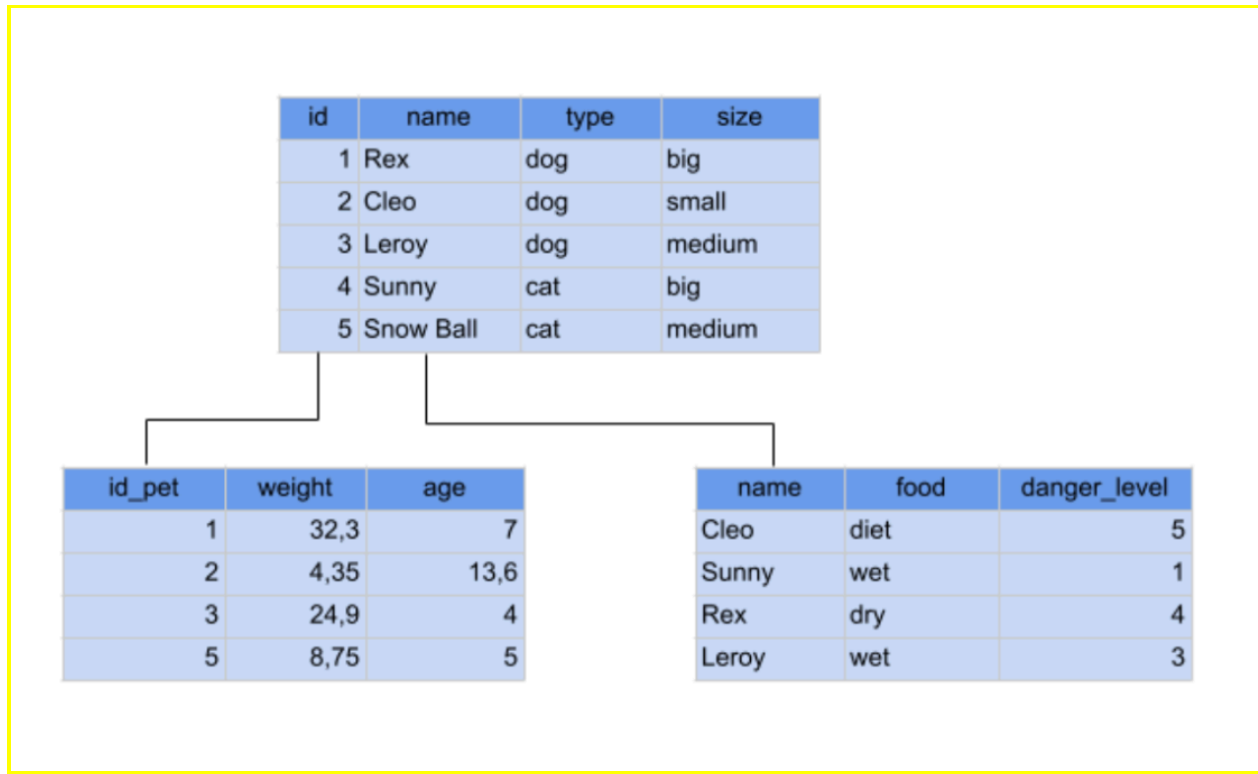
- Rows are like horizontal lines in the table. Each row holds a single record or piece of information.
- Columns are like vertical lines in the table. Each column represents a specific type of information, such as names, ages, or dates.

Database

A database is an organized collection of structured information or data, typically stored in tables, that can be easily accessed, managed, and updated. A database is usually controlled by a database management system (DBMS).

Relational Database:

- Definition: A relational database is a type of database that organizes data into tables, where each table represents a specific entity or concept, and **relationships between tables** are established using keys.



RDBMS

RDBMS stands for **Relational Database Management System**. It is a type of database **management system** that stores data in a structured format, using rows and columns in tables. RDBMSs use SQL (Structured Query Language) for managing and querying data.

The software used to store, manage, query, and retrieve data stored in a relational database is called a relational database management system (RDBMS)

RDBMS EX:

- 1.
2. PostgreSQL
3. Oracle Database

4. Microsoft SQL Server
5. SQLite
6. IBM Db2
7. MariaDB

MySQL:

MySQL is an open-source Relational Database Management System (RDBMS) that uses SQL (Structured Query Language) to manage and interact with databases. It is known for its performance, reliability, and ease of use, and it is widely used in web applications and data storage.

MySQL Workbench:

- **Purpose:** MySQL Workbench is a visual tool for database architects, developers, and DBAs. It provides a graphical interface for managing MySQL databases.



Revision

1. What is SQL and why is it important?
2. How would you define a database?
3. What makes a relational database different from other types?
4. What does RDBMS stand for and what does it do?
5. What is MySQL and what is it used for?

Answers:

1. Answer: SQL is a language used to manage and work with databases. It's important because it helps you add, update, and get data from databases

- **2. Answer:** A database is a collection of organized data. It helps you store and find information easily.
- **3. Answer:** A relational database uses tables to store data and shows how tables are connected to each other through relationships.
- **4. Answer:** RDBMS stands for Relational Database Management System. It helps you manage data stored in tables and uses SQL to perform operations on the data.
- **5. Answer:** MySQL is a popular database system that stores data in tables. It is often used for websites and applications to manage data.

TOPIC 2:

Database keys

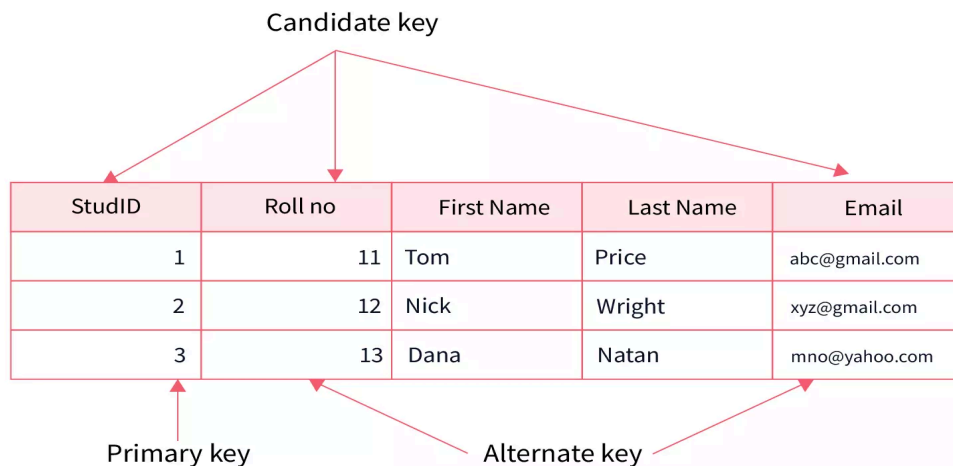
Keys are one of the basic requirements of a relational database model. It is widely used to identify the tuples(rows) uniquely in the table. We also use keys to set up relations amongst various columns and tables of a relational database.

Different Types of Database Keys

- Candidate Key
- Primary Key
- Super Key
- Alternate Key
- Foreign Key
- Composite Key

Primary Key:

- A primary key is a **unique identifier** for each record in a table.
- **It cannot contain NULL values and must be unique for each record.**
- Only one primary key can exist per table.



Candidate Key:

- A candidate key is a column or a set of columns in a table that **uniquely identifies each row** in the table.
- There can be multiple candidate keys in a table.

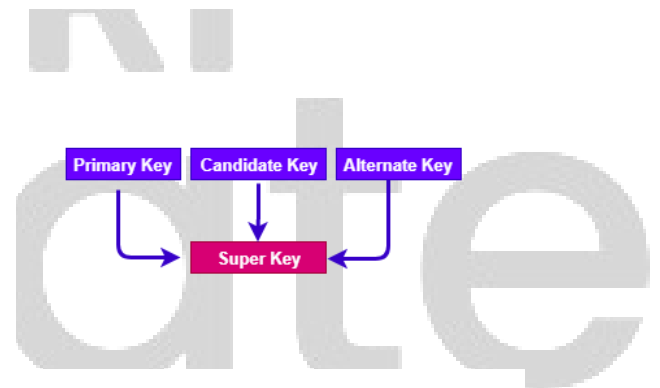
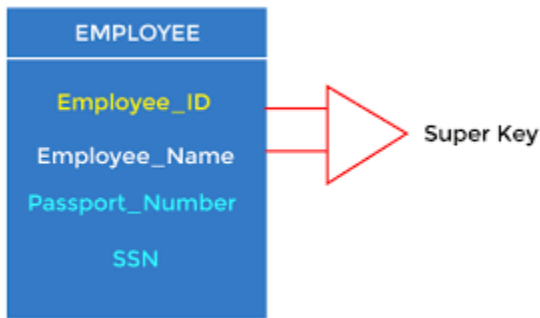
- From the set of candidate keys, one is chosen as the primary key.

Alternate Key:

- An alternate key, also known as a secondary key, is any candidate key that is not selected as the primary key.
- While it uniquely identifies rows, it is not chosen as the primary means of identification.

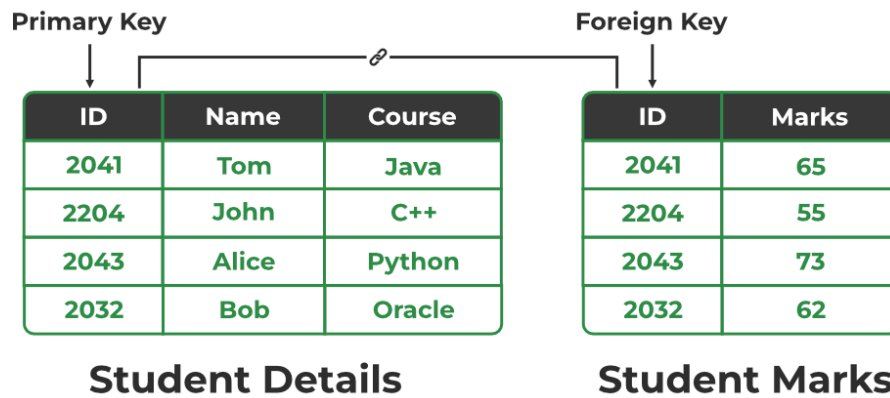
Super Key:

- A super key is a set of one or more columns that can uniquely identify each row in a table.
- It may contain more columns than necessary for uniqueness.
- Any subset of a super key is also a super key.



Foreign Key:

- A foreign key column in a table points to a column with unique values in another table (often the primary key column) to create a way of cross-referencing the two tables.
- It establishes a relationship between two tables.
- It ensures referential integrity by enforcing a link between the data in two tables.



Composite Key:

- A composite key is a key that consists of two or more columns in a table.
- Together, these columns uniquely identify a record in the table.
- Unlike primary keys, composite keys can contain NULL values, but their combination must be unique.

Composite Key			
Roll No.	Name	Age	Phone
1	Aryan	21	7491901521
2	Sachin	25	870904365
3	Prince	20	784600652
4	Anuj	21	9876534523

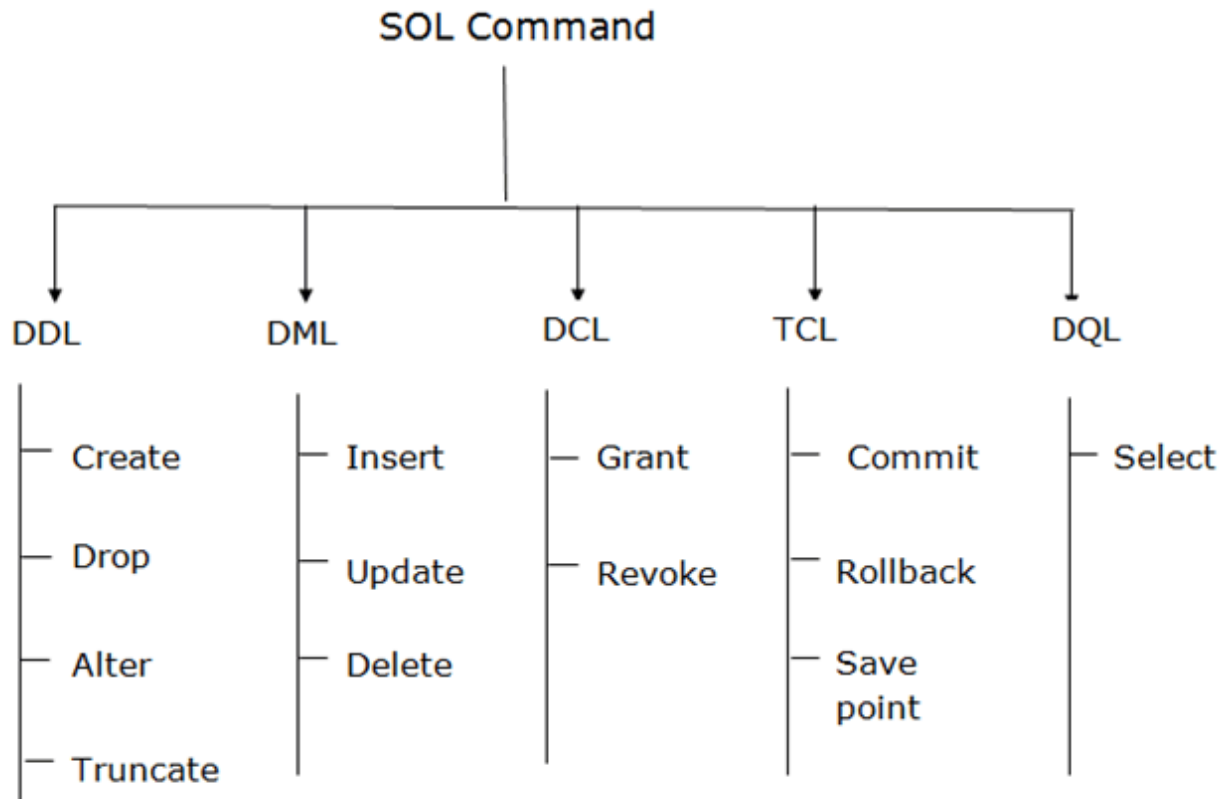
TOPIC 3:

SQL Commands (DDL, DML,DQL) & SQL Constraints

SQL Commands

In SQL (Structured Query Language), there are five main types of commands:

- DDL : Data Definition Language
- DML : Data Manipulation Language
- DQL : Data Query Language
- DCL : Data Control Language
- TCL : Transaction Control Language



Data Definition Language

- Data Definition Language consists of the SQL commands that can be used to define the Database schema.
- It simply deals with descriptions of the database schema and is used to create and modify the structure of database objects in the database.
- CREATE, ALTER, TRUNCATE and DROP are DDL commands.

1. create

```
CREATE TABLE table_name  
(  
column_Name1 data_type ( size of the column ),  
column_Name2 data_type ( size of the column ),  
column_Name3 data_type ( size of the column ),  
...  
column_NameN data_type ( size of the column )  
);
```

Example:

```
CREATE DATABASE india;  
USE india;  
CREATE TABLE kerala  
(  
Roll_No int ,  
First_Name Varchar (20) ,  
Last_Name Varchar (20) ,  
Age Int ,  
Marks Int );
```

Example 2

Rules for naming tables and columns

- The name may contain letters (A-Z, a-z), digits (0-9), under score (_) and dollar (\$) symbol.
- The name must contain at least one character. (Names with only digits are invalid).
- The name must not contain white spaces, special symbols.
- The name must not be an SQL keyword.
- The name should not duplicate with the names of other tables in the same database and with other columns in the same table.

2. Drop

DROP is a DDL command used to delete/remove the database objects from the SQL database. We can easily remove the entire table, view, or index from the database using this DDL command.

DROP DATABASE Database_Name;

DROP TABLE Table_Name;

3. ALTER Command

ALTER is a DDL command which changes or modifies the existing structure of the database, and it also changes the schema of database objects.

We can also add and drop constraints of the table using the ALTER command.

1. ALTER TABLE - ADD Column

To add a column in a table, use the following syntax:

ALTER TABLE *table_name*

```
ADD column_name datatype;
```

The following SQL adds an "Email" column to the "Customers" table:

```
ALTER TABLE Customers  
ADD Email varchar(255);
```

FIRST and AFTER clauses are used in the ALTER TABLE statement to specify the position where

a new column should be added within a table. These clauses are used when you want to specify

whether the new column should be the first column or be positioned before an existing column.

Add a new column at the beginning

```
ALTER TABLE table_name ADD COLUMN new_column_name data_type FIRST;
```

Add a new column after existing column

```
ALTER TABLE table_name ADD COLUMN new_column_name data_type AFTER  
existing_column_name;
```

2. ALTER TABLE - DROP COLUMN

To delete a column in a table, use the following syntax (notice that some database systems don't allow deleting a column):

```
ALTER TABLE table_name  
  
DROP COLUMN column_name;
```

The following SQL deletes the "Email" column from the "Customers" table:

```
ALTER TABLE Customers  
  
DROP COLUMN Email;
```

3. ALTER TABLE - RENAME COLUMN

To rename a column in a table, use the following syntax:

```
ALTER TABLE table_name
```

```
RENAME COLUMN old_name to new_name;
```

4. ADD A PRIMARY KEY:

```
ALTER TABLE table_name ADD PRIMARY KEY
```

```
(column_name);
```

5. REMOVE A PRIMARY KEY:

```
ALTER TABLE table_name DROP PRIMARY KEY;
```

4. TRUNCATE

The TRUNCATE TABLE statement is used to delete all rows from a table quickly and efficiently, without logging individual row deletions. Unlike the DELETE statement, which removes rows one by one and generates a log entry for each deleted row, TRUNCATE TABLE removes all rows in a single operation, making it faster, especially for large tables.

```
TRUNCATE TABLE table1;
```

Constraints

Constraints in SQL are rules defined on columns or tables to enforce data integrity and ensure that data values meet certain conditions. Here are some common constraints in SQL:

1. NOT NULL: Ensures that a column cannot contain NULL values.
2. UNIQUE: Ensures that all values in a column are unique.

3. **PRIMARY KEY:** Combines the NOT NULL and UNIQUE constraints. It uniquely identifies each record in a table.
4. **FOREIGN KEY:** Ensures referential integrity by enforcing a link between data in two tables.
5. **CHECK:** Ensures that all values in a column meet specified conditions.
6. **DEFAULT:** Provides a default value for a column when no value is specified.

Here's how you can define constraints in SQL:

```
CREATE TABLE db.MyTable (  
  
  ID INT PRIMARY KEY,  
  
  Name VARCHAR(50) NOT NULL,  
  
  Age INT CHECK (Age >= 18),  
  
  Email VARCHAR(100) UNIQUE,  
  
  DepartmentID INT);
```

In this example:

- The `ID` column is the primary key.
- The `Name` column cannot contain NULL values.
- The `Age` column must be greater than or equal to 18.
- The `Email` column must contain unique values.

Constraints play a vital role in maintaining data integrity and ensuring that databases remain consistent and reliable.

Create: db, table

Alter: change

drop: db, table, column-delete

Truncate: entries/ rows/ values-

DML (Data Manipulation Language)

- The SQL commands that deals with the manipulation of data present in the database belong to

DML or Data Manipulation Language and this includes most of the SQL statements.

- **INSERT, UPDATE and DELETE** are DML commands.

Note: you can perform Data Manipulation Language (DML) operations using the primary key (PK) in SQL. The primary key uniquely identifies each row in a table, so it can be used to target specific rows for INSERT, UPDATE, DELETE, or SELECT operations. Here's how you can use the primary key in different DML operations:

1. **INSERT:** When inserting new rows into a table, you typically don't include the primary key column in the VALUES list because it's often an auto-incremented or generated value. Instead, the database system automatically assigns a unique value to the primary key column for each new row.

```
INSERT INTO table_name (column1, column2, ...)
VALUES (value1, value2, ...);
```

2. **UPDATE:** When updating existing rows, you can use the primary key in the WHERE clause to target specific rows for modification.

```
UPDATE table_name  
SET column1 = value1, column2 = value2, ...  
WHERE primary_key_column = value;
```

3. **DELETE:** When deleting rows from a table, you can use the primary key in the WHERE clause to specify which rows to remove.

```
DELETE FROM table_name  
WHERE primary_key_column = value;
```

4. **SELECT:** When retrieving data from a table, you can use the primary key to fetch specific rows or to join tables based on the primary key column.

```
SELECT * FROM table_name  
WHERE primary_key_column = value;
```

1.DML - Insert into

- The INSERT INTO statement in MySQL is used to insert new records (rows) into a table.
- It allows you to add data to a table by specifying the values you want to insert for each column or by providing values for all columns in the order they are defined in the table.

```
INSERT INTO table_name (column1, column2, column3, ...)  
VALUES (value1, value2, value3, ...);
```

example1:l

```
INSERT INTO india5.kerala  
VALUES(6,"CHARLES","BABAGE", 12,"DOCT",99,"hai@gmail.com"),  
(2,"CHARLES","BABAGE", 12,"DOCT",99,"hai@gmail.com"),  
(3,"CHARLES","BABAGE", 12,"DOCT",99,"hai@gmail.com");
```

Example2:

```
INSERT INTO india5.kerala (Roll_No, Marks, Age, First_Name)  
VALUES (10,99,8,"ki;i");
```

2, DQL SELECT

DQL (Data Query Language)

- Data Query Language (DQL) is a subset of SQL (Structured Query Language) that specifically deals with the retrieval of data from a relational database.
- The primary command associated with DQL is the SELECT statement.

1. Select All Columns from Table:

```
SELECT * FROM college.sqldataset;
```

This query retrieves all columns and rows from the "sqldataset" table in the "college" schema.

2. Select Specific Columns from Table:

```
SELECT gender, lunch FROM college.sqldataset;
```

This query retrieves only the "gender" and "lunch" columns from the "sqldataset" table.

3. Filter Rows Based on Condition:

```
SELECT gender, lunch FROM college.sqldataset WHERE gender = 'female';
```

This query retrieves the "gender" and "lunch" columns for rows where the "gender" column has the value "female".

4. Select Distinct Values:

```
SELECT DISTINCT gender, lunch FROM college.sqldataset;
```

This query retrieves unique combinations of values for the "gender" and "lunch" columns from the "sqldataset" table.

5. Filter Rows Using LIKE Operator:

```
SELECT gender, lunch FROM college.sqldataset WHERE lunch LIKE '%sta%';
```

This query retrieves the "gender" and "lunch" columns for rows where the "lunch" column contains the substring "sta".

Pattern	Meaning
'a%'	Match strings that start with 'a'
'%a'	Match strings with end with 'a'
'a%t'	Match strings that contain the start with 'a' and end with 't'.
'%wow%'	Match strings that contain the substring 'wow' in them at any position.
'_wow%'	Match strings that contain the substring 'wow' in them at the second position.
'_a%'	Match strings that contain 'a' at the second position.
'a_ _%'	Match strings that start with 'a' and contain at least 2 more characters.

6. Order Rows by Column:

```
SELECT * FROM college.sqldataset ORDER BY gender ASC;
```

This query retrieves all columns and rows from the "sqldataset" table and orders them in ascending order based on the "gender" column.

3. DML Update

1. Update a Single Row (reg_no = 1):

```
UPDATE college.sqldataset  
SET gender = 'Vegetarian'  
WHERE reg_no = 1;
```

- This query updates the "gender" column to 'Vegetarian' for the row where the "reg_no" column equals 1.
- The WHERE clause ensures that only the row with "reg_no" equal to 1 is updated.

2. Update Multiple Rows (reg_no IN (1,2,3,4)):

```
UPDATE college.sqldataset  
SET gender = 'Vegetarian'  
WHERE reg_no IN (1,2,3,4);
```

- This query updates the "gender" column to 'Vegetarian' for the rows where the "reg_no" column matches any value in the list (1, 2, 3, or 4).
- The IN operator allows us to specify multiple values to match against.

Notes:

- The UPDATE statement modifies existing data in a table by changing the values of specified columns.

4. DML Commands - DELETE

- The DELETE statement is used to delete existing records in a table.
- Be careful when deleting records in a table! Notice the WHERE clause in the DELETE statement.

The WHERE clause specifies which record(s) should be deleted. If you omit the WHERE clause,
all records in the table will be deleted!

```
DELETE FROM table_name WHERE condition;
```

Delete All Records:

```
DELETE FROM table_name;
```

Can you explain the differences between the SQL commands DROP (DDL) , DELETE(DML), and TRUNCATE(DDL)?

Answer:

- "DROP" is used to **permanently remove objects** like tables, views, or indexes from the database. It's a DDL command and results in the complete elimination of the object and its definition. Example: `DROP TABLE TableName;`
- "DELETE" is used to **remove specific rows** from a table based on specified criteria. It's a DML command and leaves the table structure intact while removing selected data. Example: `DELETE FROM TableName WHERE condition;`
- "TRUNCATE" is used to **remove all rows** from a table in a single operation, effectively resetting the table to its initial state. It's faster than DELETE as it doesn't generate individual row-wise transactions for deletion, but it also doesn't allow specifying conditions for which rows to delete. It's also a DDL command. Example: `TRUNCATE TABLE TableName;`

Codes :

```
use students;
```

```
select * from 10a;
```

```
alter table 10a add primary key (roll);
```

```
insert INTO 10a VALUES(8,"lakshmi",16,"female",89,99);
```

```
insert INTO 10a  
VALUES(9,"madhav",16,"male",89,99),(10,"hai",15,"male",67,98);
```

```
select * from 10a;
```

```
INSERT INTO 10a (roll, name, gender, math_mark)  
  
VALUES (11,"hello","female",100);
```

```
select * from 10a; -- dql
```

```
select name from 10a;
```

```
select roll, name from 10a;
```

```
select * FROM 10a WHERE gender = 'female';
```

```
SELECT * FROM 10a WHERE sci_mark<90 ;
```

```
SELECT DISTINCT gender,age FROM 10a;
```

```
SELECT * FROM 10a WHERE name LIKE 'a%';
```

```
SELECT * FROM 10a WHERE name LIKE '%hu';
```

```
SELECT * FROM 10a WHERE name LIKE '%s%';
```

```
SELECT * FROM 10a ORDER BY gender ASC;
```

```
SELECT * FROM 10a ORDER BY math_mark ASC;
```

```
SELECT * FROM 10a ORDER BY math_mark desc;
```

```
UPDATE 10a
```

```
SET name = 'honey'
```

```
WHERE roll = 10;
```

```
select * from 10a;
```

```
UPDATE 10a
```

```
SET gender = 'Vegetarian'
```

```
WHERE roll IN (1,2,3,4);
```

```
DELETE FROM 10a WHERE roll in (3,4);
```

Drop: db, table, columns

Delete: specific rows

Truncate: all rows

CLAUSE:

<https://drive.google.com/file/d/1avJSj1v96ixoDSLISqpwwSKt0E9xc7Gx/view?usp=sharing>

CONSTRAINS:

1. Table Creation Without Constraints

```
CREATE TABLE employees_no_constraints (  
    id INT,  
    first_name VARCHAR(50),  
    last_name VARCHAR(50),  
    email VARCHAR(100),  
    hire_date DATE,  
    salary DECIMAL(10, 2),  
    department_id INT,  
);
```

2. Table Creation With Constraints

```
CREATE TABLE employees_with_constraints (  
    id INT AUTO_INCREMENT PRIMARY KEY,  
    first_name VARCHAR(50) NOT NULL,  
    last_name VARCHAR(50) NOT NULL,  
    email VARCHAR(100) UNIQUE NOT NULL,  
    hire_date DATE NOT NULL,  
    salary DECIMAL(10, 2) CHECK (salary > 0),  
    department_id INT NOT NULL,  
    AGE INT CHECK (AGE >= 18)  
);
```

Comparison:

Aspect	Without Constraints	With Constraints
Primary Key	<code>`id INT`</code>	<code>`id INT AUTO_INCREMENT PRIMARY KEY`</code>
NOT NULL	<code>`first_name VARCHAR(50)`</code>	<code>`first_name VARCHAR(50) NOT NULL`</code>
UNIQUE	<code>`email VARCHAR(100)`</code>	<code>`email VARCHAR(100) UNIQUE NOT NULL`</code>
CHECK	<code>`salary DECIMAL(10, 2)`</code>	<code>`salary DECIMAL(10, 2) CHECK (salary > 0)`</code>
Foreign Key	<code>`department_id INT`</code>	<code>`department_id INT NOT NULL,` <code>`FOREIGN KEY (department_id) REFERENCES departments(id)`</code></code>